# How to create a custom Kernel for a Gaussian Process Regressor in scikit-learn?

Asked 6 years, 9 months ago  Modified 5 months ago  Viewed 8k times

I am looking into using a GPR for a rather peculiar context, where I need to write my own Kernel. However I found out there's no documentation about how to do this. Trying to simply inherit from `Kernel` and implementing the methods `__call__`, `get_params`, `diag` and `is_stationary` is enough to get the fitting process to work, but then breaks down when I try to predict y values and standard deviations. What are the necessary steps to build a minimal but functional class that inherits from `Kernel` while using its own function? Thanks!

**scikit-learn** Edit tags

Share Edit Follow Close Flag

asked Mar 8, 2018 at 17:01
Okarin
954 1 11 22

Off-topic here, but have you seen scikit-learn.org/stable/modules/...? It also might be helpful to look at the source of kernels in the library, e.g. github.com/scikit-learn/scikit-learn/blob/0.19.1/sklearn/... – Danica Mar 8, 2018 at 17:06

Yeah, I've seen it. I've also found an example on Github of someone who created new custom Kernel classes: github.com/scikit-learn/scikit-learn/blob/0.19.1/sklearn/... Problem is, I would like very clear definitions of how the interface should be implemented. Especially regarding gradients and such, I'd hate making a mistake that makes the thing still work but then makes the numbers wrong. – Okarin Mar 8, 2018 at 17:13

@Okarin Unit tests are your friend. – Sycorax Mar 8, 2018 at 17:15

As an aside, if you haven't seen it, github.com/GPflow/GPflow is possibly a better option; you don't have to code gradients manually. github.com/SheffieldML/GPy is also more flexible than scikit-learn's implementation. If not, re @Sycorax's suggestion, `scipy.optimize.check_grad` might be useful. – Danica Mar 8, 2018 at 17:16

I don't have a problem calculating manually the analytical form of the gradients, I'm just worried about the interface of those methods and didn't want to just discover things by trial-and-error. I would have liked to know if there was some clear guide on how to build my own Kernel class, what methods MUST be overridden, etc. – Okarin Mar 9, 2018 at 11:44

## 1 Answer

Sorted by: Reset to default  Date modified (newest first)

Depending on how exotic your kernel will be, the answer to your question may be different.

I find the implementation of the RBF kernel quite self-documenting, so I use it as reference. Here is the gist:

```
class RBF(StationaryKernelMixin, NormalizedKernelMixin, Kernel):
    def __init__(self, length_scale=1.0, length_scale_bounds=(1e-5, 1e5)):
        self.length_scale = length_scale
        self.length_scale_bounds = length_scale_bounds

    @property
    def hyperparameter_length_scale(self):
        if self.anisotropic:
            return Hyperparameter("length_scale", "numeric",
                                  self.length_scale_bounds,
                                  len(self.length_scale))
        return Hyperparameter(
            "length_scale", "numeric", self.length_scale_bounds)

    def __call__(self, X, Y=None, eval_gradient=False):
        # ...
```

As you mentioned, your kernel should inherit from Kernel, which requires you to implement `__call__`, `diag` and `is_stationary`. Note, that `sklearn.gaussian_process.kernels` provides `StationaryKernelMixin` and `NormalizedKernelMixin`, which implement `diag` and `is_stationary` for you (cf. RBF class definition in the code).

You should not overwrite `get_params`! This is done for you by the Kernel class and it expects scikit-learn kernels to follow a convention, which your kernel should too: *specify your parameters in the signature of your constructor as keyword arguments* (see `length_scale` in the previous example of RBF kernel). This ensures that your kernel can be copied, which is done by `GaussianProcessRegressor.fit(...)` (this could be the reason that you could not predict the standard deviation).

At this point, you may notice the other parameter `length_scale_bounds`. That is only a constraint on the actual hyper parameter `length_scale` (cf. constrained optimization). This brings us to the fact, that you need to also declare your hyper parameters, that you want optimized and need to compute gradients for in your `__call__` implementation. You do that by defining a property of your class that is prefixed by `hyperparameter_` (cf. `hyperparameter_length_scale` in the code). Each hyper parameter that is not fixed (fixed = `hyperparameter.fixed == True`) is returned by `Kernel.theta`, which is used by GP on `fit()` and to compute the marginal log likelihood. So this is *essential* if you want to fit parameters to your data.

One last detail about `Kernel.theta`, the implementation states:

> Returns the (flattened, log-transformed) non-fixed hyperparameters.

So you should be careful with 0 values in your hyper parameters as they can end up as np.nan and break stuff.

I hope this helps, even though this question is already a bit old. I have actually never implemented a kernel myself, but was eager to skim the sklearn code base. It is unfortunate that there are no official tutorials on that, the code base, however, is quite clean and commented.

Share Edit Follow Flag

edited Jul 1 at 19:55
Till Hoffmann
9,827 7 50 67

answered Feb 13, 2019 at 15:03
mbornstein
264 3 7

1 Thank you for the references. Highlighting important aspects of the source code certainly is helpful, although implementing a concrete example (e.g. tanh kernel) would be generally helpful for everyone to follow the steps one-by-one. I'm still trying to work on a solution that can be shared, but the devil is in the details and it's easy as OP mentioned to create small yet costly errors. – Mathews24 Feb 13, 2019 at 17:46

I see what you mean. Is there any concrete problem that you are working on, that you would like help with? – mbornstein Feb 13, 2019 at 21:03

This would be one related example where implementing a tanh kernel in 2-dimensions is a current project. – Mathews24 Feb 14, 2019 at 11:55

Thanks! This is really helpful! It'd be nice to see something like this in the official docs. – Okarin Feb 14, 2019 at 18:00